



Restaurant Management System

An Object-Oriented Programming project built with c++ simulating real-world restaurant operations through clean class design.

Meet the Team

Maryam Al-Saadi 24-O563

Maryam Albalshi 24-O324

Noura Al-Mahmoodi 24-O259

Ghudaina Al-Issaei 24-O181

Jannah Alrizaiqi 24-O351

Table of Contents

O1

System Description

O3

Main Classes

O5

Relationships Between Classes

O2

System Features

O4

Class Responsibilities

O6

Conclusion & OOP Concepts

System Description



This system is designed to manage a restaurant's core operations by providing **menu management**, **order handling**, and **payment processing** — all simulated through object-oriented design.



Menu Handling

Display items with prices



Order Management

Track and calculate totals



Payment Processing

Support cash and card

System Features



Display Menu

Show all available food items with their prices in a structured format.



Place Orders

Allow customers to select and add multiple items to their order.



Calculate Totals

Automatically compute the total cost of all items in an order.



Payment Methods

Support both cash and card payment options for flexibility.



Manage Multiple Orders

Handle several customer orders simultaneously within the system.

Main Classes

The system is built around **five core classes**, each with a single, well-defined responsibility.



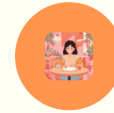
MenuItem

Represents a single food item with a name and price.



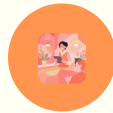
Order

Stores a customer's selected items and calculates the total.



Customer

Represents a customer who places and owns an order.



Restaurant

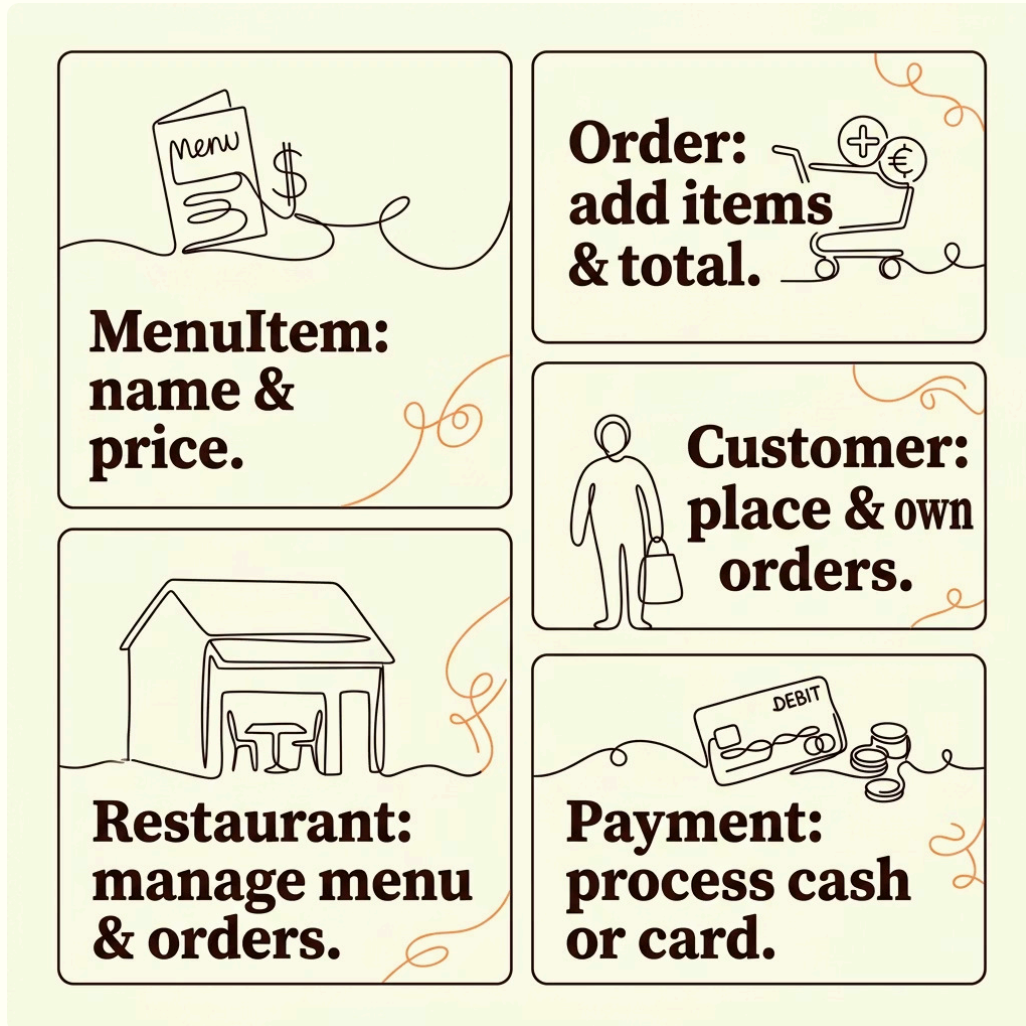
Manages the menu and the list of all active orders.



Payment

Handles payment processing for cash or card transactions.

Class Responsibilities



Each class follows the **Single Responsibility Principle** — one class, one job. This keeps the code clean, testable, and easy to extend.

📌 **Key Insight:** MenuItem and Order are the most reused classes — every transaction flows through them.

Relationships Between Classes



Composition

Order contains multiple **MenuItem** objects — an order is made up of items. If the order is deleted, its items are removed too.



Association

Customer has an **Order** — a customer places and owns an order. This is a "has-a" relationship.



Aggregation

Restaurant manages both **MenuItem** and **Order** lists — it holds references but doesn't own them exclusively.



Inheritance & Polymorphism

Payment uses a parent class with subclasses for **CashPayment** and **CardPayment**, demonstrating runtime polymorphism.

